

Input System: Event-Driven Pipeline & Application Systems Extension Point

Design document — not yet implemented. Per this repository's read-only policy, this is a proposal for a human to build; Claude has not touched any source file.

Executive summary

The engine already has one working example of the shape this design reuses: the entity manager's component pipeline (`EntityManager` → `ComponentDispatcher` → N `IComponentRegistrar` s). Input needs the same shape for the same reason components do — multiple independent producers (keyboard today, gamepad soon) need to resolve into one shared vocabulary (`Action`) without the aggregator knowing about each producer's internals.

This document proposes:

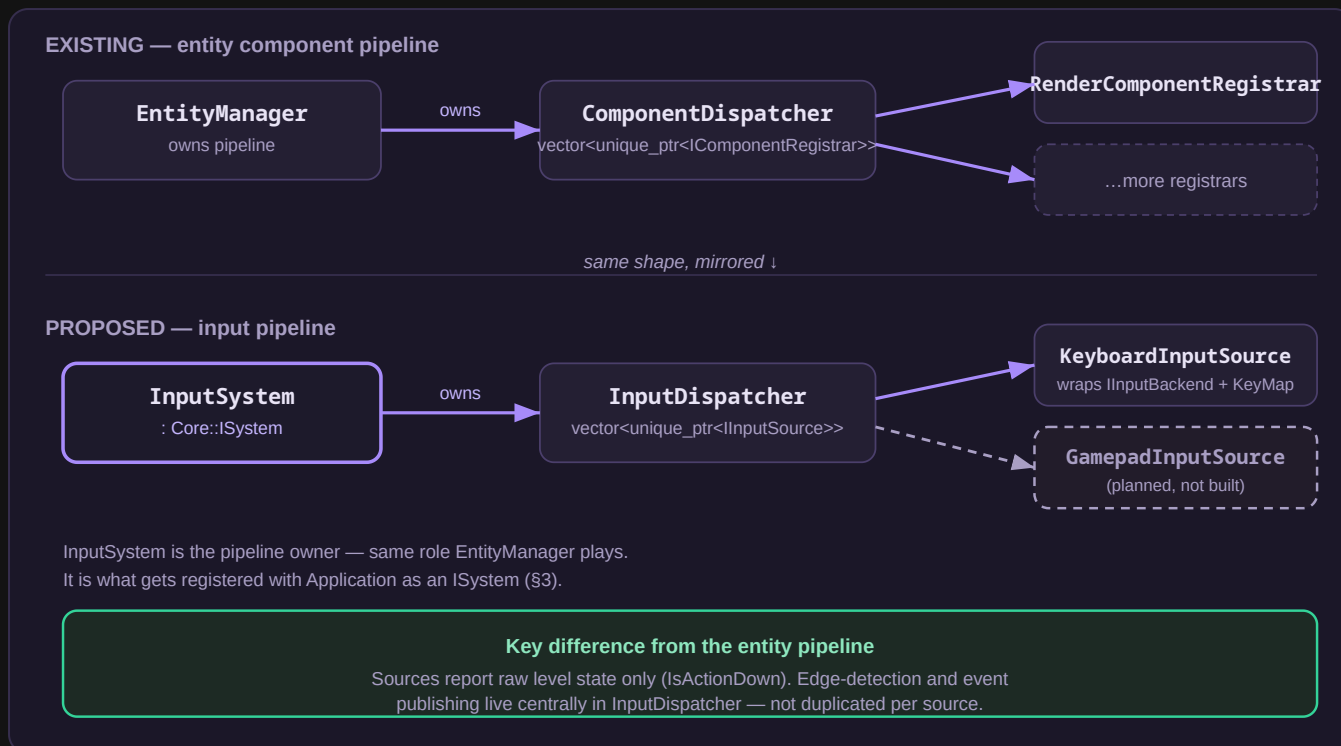
1. An **input pipeline** — `IInputSource` implementations (one per device) feed an `InputDispatcher` , which aggregates per- `Action` state, edge-detects press/release transitions, and publishes them on the existing `IEventBus` .
2. An **ISystem extension point on Application** — a small, explicit, app-global list of update-driven systems, run once per frame *before* the scene manager, so that synchronously-dispatched events (like input) reach their subscribers in the same frame they occur.

Net effect: `SnakeGame::GameState::Update()` stops polling `IsActionPressed()` every frame (removing the `MAIN QUEST` TODO already sitting at `gameplay-state.cpp:45-63`) and instead subscribes once to `InputActionPressedEvent` .

1. Why mirror the entity pipeline pattern

The entity pipeline pattern earns its complexity because it solves a specific problem: an **open-ended set of producers, each opting into a shared concern, aggregated by one dispatcher that doesn't know their concrete types**. For entities that's "component type → registrar." For input, the same shape shows up once a second device is in scope: "input device → resolves into `Action` state."

Today there is one device (keyboard) and no dispatcher — `InputSystem` polls `IInputBackend` directly. That's fine in isolation, but a planned gamepad source means the aggregation logic (OR-ing multiple devices' state per `Action` , edge-detecting once, not per device) needs a home now, or it gets bolted on awkwardly later.



■ owns (`unique_ptr`) --- planned / not-yet-built

2. Input pipeline types

```
// core/input/i-input-source.hpp
namespace Core::Input {
class IInputSource {
public:
    virtual ~IInputSource() = default;
    virtual bool IsActionDown(Engine::Input::Action action) const = 0;
};
} // namespace Core::Input
```

`IInputSource` reports **level state only** — no press/release logic. That logic is centralized once in the dispatcher instead of being reimplemented per device (today's `InputSystem::IsActionPressed` derives edges from raylib's own per-frame `IsKeyPressed`, which won't exist in the same form on a gamepad backend — better to not depend on each device having edge-detection built in).

```
// core/input/i-input-dispatcher.hpp
namespace Core::Input {
class IInputDispatcher {
public:
    virtual ~IInputDispatcher() = default;
    virtual void AddSource(std::unique_ptr<IInputSource> source) = 0;
    virtual void Run() = 0;
};
} // namespace Core::Input
```

```
// engine/input/input-dispatcher.hpp / .cpp
namespace Engine::Input {
class InputDispatcher final : public Core::Input::IInputDispatcher {
public:
    InputDispatcher(Core::Events::IEventBus& eventBus);
    void AddSource(std::unique_ptr<Core::Input::IInputSource> source) override;
    void Run() override;
    bool IsActionDown(Action action) const; // pull-query escape hatch, see §4

private:
    std::vector<std::unique_ptr<Core::Input::IInputSource>> sources;
    std::array<bool, static_cast<size_t>(Action::Count)> lastFrameDown = {};
    Core::Events::IEventBus& eventBus;
};
} // namespace Engine::Input
```

`Run()` , per `Action` :

1. `isDown = OR` across every source's `IsActionDown(action)` .
2. Compare against `lastFrameDown[action]` .
3. On `false → true` , `eventBus.Publish(InputActionPressedEvent{action})` . On `true → false` , `eventBus.Publish(InputActionReleasedEvent{action})` .
4. `lastFrameDown[action] = isDown` .

```
// engine/input/sources/keyboard-input-source.hpp
namespace Engine::Input::Sources {
class KeyboardInputSource final : public Core::Input::IInputSource {
public:
    KeyboardInputSource(Core::Input::IInputBackend& backend, const KeyMap& keyMap);
    bool IsActionDown(Action action) const override;

private:
    Core::Input::IInputBackend& backend;
    KeyMap keyMap;
};
} // namespace Engine::Input::Sources
```

This is today's `InputSystem::IsActionDown` body, moved down one layer unchanged.

`GamepadInputSource` gets the same shape later — that's the whole point of factoring this out now rather than after the second device shows up.

```
// engine/input/input-system.hpp
namespace Engine::Input {
class InputSystem final : public Core::ISystem {
public:
    InputSystem(Core::Input::IInputBackend& backend, Core::Events::IEventBus& eventBus, const
KeyMap& keyMap);
    void OnUpdate(float deltaTime) override;    // -> dispatcher.Run()
    void OnDebugUpdate() const override;        // optional: overlay current action state
    void OnDebugRender() const override;

private:
    InputDispatcher dispatcher;
};
} // namespace Engine::Input
```

`InputSystem` is the pipeline **owner** — the same role `EntityManager` plays over its two pipelines. It's also the thing that becomes an `ISystem` and gets registered with `Application` (§5).

3. Events

`IEventBus::Publish<TEvent>` dispatches by `typeid` against `std::any` — nothing requires events to derive from `Core::Events::IEvent`. (That marker interface exists in the codebase but nothing currently implements it; every event type so far, including `window-resized-event.hpp` and `apple-collected-events.hpp`, is an empty stub.) Plain structs are enough:

```
// core/events/input/input-action-pressed-event.hpp
namespace Core::Events::Input {
struct InputActionPressedEvent {
    Engine::Input::Action action;
};
} // namespace Core::Events::Input

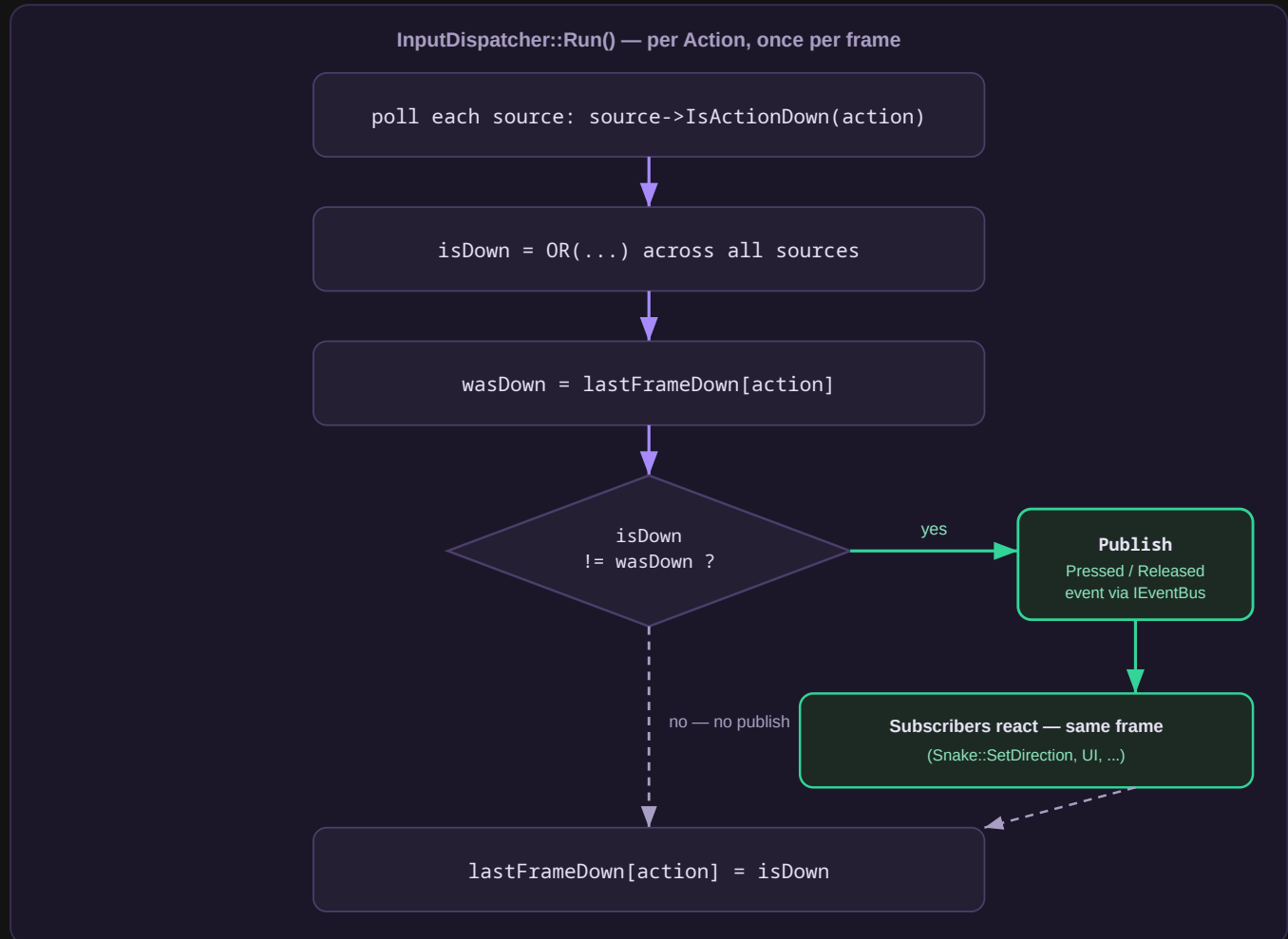
// core/events/input/input-action-released-event.hpp
namespace Core::Events::Input {
struct InputActionReleasedEvent {
    Engine::Input::Action action;
};
} // namespace Core::Events::Input
```

These fill in the two existing empty stubs at `core/events/input/key-pressed-event.hpp` / `key-released-event.hpp` — but named after `Action`, not `KeyCode`. That naming choice matters: a rebind, or a second device entirely, never touches subscriber code, because subscribers only ever hear about the semantic action, never the physical key or button.

Cleanup note. There are currently two empty stub locations for these events — `core/events/input/` and `engine/events/input/`. Recommend consolidating to `core/events/input/` only, consistent

with `core/events/window/` already holding the window event types at the `core` layer.

4. Per-frame dispatch & edge-detection flow



The dispatcher never needs to know *which* source flipped the bit — only that the aggregate changed. That's what makes adding `GamepadInputSource` later a pure addition: it plugs into step 1's OR, nothing downstream changes.

`InputDispatcher::IsActionDown(action)` (the pull-query mentioned in §2) is a thin read of the same aggregated `isDown` computed in step 2, exposed for callers that genuinely want level-based polling (e.g. a UI element checking "is Confirm currently held") rather than subscribing to edges. Snake's own movement doesn't need it — see §6.

5. Application systems extension point

`Core::IDependencyInjector` binds one implementation per interface type — there is no "resolve all registered `ISystem`" query. So the systems list can't be assembled by the container; `Application` has to

build it explicitly, the same way it already hand-resolves `window`, `renderer`, `input`, etc. in `Initialize()`.

```
// core/systems/i-system.hpp
namespace Core {
class ISystem : public Core::IObservable, public Core::Debug::IDebugable {};
} // namespace Core
```

No new methods — `ISystem` just names the combination of hooks (`OnUpdate`, `OnDebugUpdate`, `OnDebugRender`) that `Application` already knows how to drive, the same combination `IEntityManager` implements today.

Wiring — RegisterDependencies() / Initialize()

```
auto input = std::make_shared<InputSystem>(*inputBackend, *eventBus, keyMap);
this->RegisterSystem(input);
```

`Application::systems`

`vector<shared_ptr<ISystem>>` — app-global, survives scene/state changes

Frame loop — Application::OnUpdate(deltaTime)

```
for (auto& system : this->systems) system->OnUpdate(deltaTime);
1st — includes InputSystem, publishes this frame's action events
```

```
if (sceneManager) sceneManager->OnUpdate(deltaTime);
```

2nd — scene/state logic reads this frame's already-published events

Reversing this order delays every press by one frame — `IEventBus::Publish` is synchronous, no queue.

`InputSystem` needs `IInputBackend&` and `IEventBus&`, both already resolved by `Application::Initialize()` before systems are constructed, so no new DI plumbing is required — just an explicit construction + `RegisterSystem` call alongside the existing hand-wiring block.

6. Downstream effect on `SnakeGame`

`SnakeGame::GameContext` (`gameplay-state-machine.hpp:22`) currently holds `unique_ptr<Engine::Input::InputSystem> input` as a per-state polling handle. Once `InputSystem` is app-global and event-driven, that field goes away. `GameState` (or `Snake` itself) subscribes once — e.g. in `Enter()` or construction — instead of the polling block at `gameplay-state.cpp:45-63`:

```
// illustrative – not applied
this->eventBus.Subscribe<InputActionPressedEvent>([this](const auto& e) {
    switch (e.action) {
        case Action::MoveLeft:  this->snake->SetDirection({-1, 0}); break;
        case Action::MoveRight: this->snake->SetDirection({1, 0});  break;
        case Action::MoveUp:    this->snake->SetDirection({0, -1}); break;
        case Action::MoveDown:  this->snake->SetDirection({0, 1});  break;
        default: break;
    }
});
```

Why edge-triggered is correct here, not a compromise: Snake only cares about direction *changes* — it never needs to re-apply the same direction every frame the key stays held. That's exactly what `InputActionPressedEvent` gives for free. A future held/level-based action (e.g. a "boost" that only applies while a key is down) is the one case that should use `InputDispatcher::IsActionDown()` as a pull-query instead — no third event type needed.

7. Summary — design decisions and what they resolve

Decision	Resolves	Rationale
<code>IInputSource</code> reports level state only, no per-source edge logic	Duplicated press/release logic across devices	Raylib gives free edge detection; a gamepad backend may not. Centralizing in the dispatcher means every device only needs to answer "is this down right now."
<code>InputDispatcher</code> aggregates via OR before edge-detecting	Multiple devices publishing conflicting/duplicate events for one logical press	Dispatcher sees one boolean per <code>Action</code> , not N per device — matches how <code>ComponentDispatcher</code> sees one component stream, not N per registrar.
Events keyed by <code>Action</code> , not <code>KeyCode</code>	Rebinding / new devices breaking subscriber code	Subscribers only ever hear semantic actions.
Plain structs for events, no <code>IEvent</code> base	Following existing (unused) pattern needlessly	<code>IEventBus</code> dispatches by <code>typeid</code> ; nothing in the codebase actually requires <code>IEvent</code> .
<code>ISystem</code> list is app-global, owned by <code>Application</code>	Needing to design scene-transition ownership for systems right now	Matches how input actually behaves — always live, not scene-scoped. Deferred, not rejected: revisit if a system needs scene-scoped lifetime later.
Systems run before <code>SceneManager->OnUpdate()</code>	Input events arriving one frame late	<code>IEventBus::Publish</code> is synchronous — publisher and subscriber must run in dispatch order within the same frame.
Systems list assembled explicitly in <code>RegisterDependencies()</code> / <code>Initialize()</code> , not DI-resolved	<code>IDependencyInjector</code> has no "resolve all X" query	Matches existing hand-wiring pattern for <code>window</code> / <code>renderer</code> /etc. — no new DI capability needed.

8. Suggested adoption order

1. Add `Core::ISystem` + the `systems` vector and `RegisterSystem` helper on `Application`; reorder `OnUpdate` (systems, then scene manager). No behavior change yet — empty list.
2. Add `Core::Input::IInputSource`, `IInputDispatcher`, `Engine::Input::InputDispatcher`.

3. Add `KeyboardInputSource` , wrapping today's `InputSystem` polling logic verbatim.
4. Fill in `InputActionPressedEvent` / `InputActionReleasedEvent` (consolidate the duplicate empty stub locations down to `core/events/input/` while here).
5. Rework `InputSystem` into the `ISystem` -implementing pipeline owner; register it in `Application::RegisterDependencies()` .
6. Migrate `GameState` / `Snake` from polling `GameContext::input` to subscribing to `InputActionPressedEvent` ; remove the `GameContext::input` field and the TODO block at `gameplay-state.cpp:45-63` .
7. When a gamepad backend exists: add `GamepadInputSource` , register it alongside `KeyboardInputSource` in `InputSystem` 's constructor. No other code changes required — this is the step the whole design exists to make cheap.